Software Project Estimation

Rohayanti Hassan
Faculty of Computing
Universiti Teknologi Malaysia

This slide contains material that is copied from the book Agile Project Manager_v2_handbook by Agile Business Consortium. All rights reserved. No part of this material may be reproduced or transmitted in any form or by any means without the prior written permission of the copyright owner. This material is used here for educational purposes under the doctrine of fair use, with the intention to support learning and teaching.

For more information, please refer to the original source:
Book Title: Agile Project Manager_v2_handbook
Author: Agile Business Consortium
Publisher: Buckland Media Group Limited
Year of Publication: 2017
Please consult the book for further details and complete content.

Outline

- Understanding Software Effort Estimation Basics
- Factors Affecting Software Development Costs
- Proven Techniques for Estimating Project Budgets
- Top-Down Estimation
- Bottom-Up Estimation

Learning Outcomes

By the end of this chapter, you will be able to:

- Understand the fundamentals of software effort and cost estimation
- Explain key estimation concepts such as effort, cost, size, and complexity
- Compare traditional estimation models with agile-based estimation approaches
- Apply estimation techniques such as T-Shirt,Velocity,Function Point and others.
- Recognize factors that influence estimation accuracy
- Appreciate the importance of estimation in planning, budgeting, and monitoring software projects

Understanding Software Effort Estimation Basics



What is Software Effort Estimation?

Software Effort Estimation refers to predicting the amount of work, time, and resources required to develop a software product.

- Helps determine project feasibility
- Supports budgeting and scheduling
- Essential for planning releases in both traditional and agile projects
- Mentioned as a critical early-stage activity in research (Rivera Ibarra et al., 2024)



Purpose of Software Effort Estimation



Estimation provides a foundation for making informed decisions about project planning and delivery. It reduces uncertainty and helps teams prepare for the work ahead.

- Plan project timelines
- Allocate team members and resources
- Estimate cost and effort
- Reduce project risks and uncertainties
- Support decision-making for contracts and budgeting

(Reference: Mike Cohn, Agile Estimating and Planning)

Importance of Software Estimation



Accurate estimation improves planning and ensures projects deliver value within realistic constraints. It helps both teams and stakeholders set expectations.

- Prevents overruns in cost and schedule
- Enables better resource planning
- Helps manage stakeholder expectations
- Supports delivery of high-value features

Challenges in Estimation

Software development involves uncertainty, changing requirements, and human judgement —all of which make estimation difficult.



- Requirements may be unclear early
- Technical complexity is hard to predict
- Human bias affects estimates
- Teams may lack historical data
- Agile projects constantly evolve

Effort vs Duration vs Cost

Although related, effort, duration, and cost measure different aspects of a project and should not be confused.

- Effort: Human labor required
- Duration: Calendar time including delays
- Cost: Monetary value based on effort and resources
- Effort does not always equal duration



Factors Affecting Software Development Costs



What Effects Software Development Costs?

Software development costs are influenced by many internal and external factors. These factors impact effort, timeline, resources, and estimation accuracy across projects.

- Product characteristics
- Project environment
- Technical complexity
- Human and team factors
- Organizational maturity
- Lessons from empirical studies



Product Factors



Product-related attributes significantly influence how much effort a project requires. Larger or more complex products naturally demand more time and resources.

- System size (SLOC, FP, Story Points)
- Functional complexity (features, interfaces)
- Required performance, reliability, and security
- Number of modules or integrations

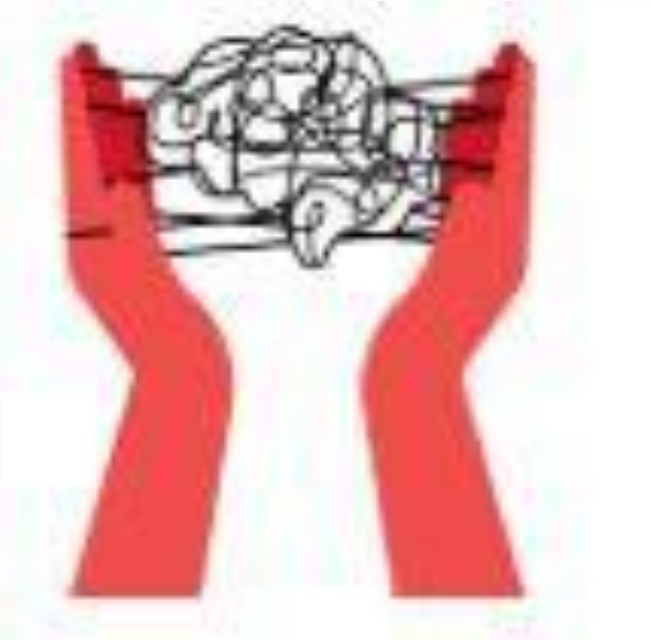
Project Environment Factors

The environment surrounding the project also affects cost. These conditions shape how smoothly development can proceed.

- Tools, frameworks, and technologies used
- Agile vs traditional environment difference
- Availability of domain experts
- Requirements clarity or volatility
- Contract type (flexible vs fixed) impacts estimation variation



Technical Complexity Modeling



Technical complexity is a major cost driver because complex software demands more effort in design, coding, testing, and integration.

- Complex algorithms or data structures
- Multiple system interfaces
- Security and compliance needs
- Integration with legacy systems
- Hidden complexity increases effort (Scrum SLR finding)

Human & Team Factors

People-related factors play a major role. Skills, communication, and team structure have direct effects on cost and productivity.

- Team experience influences accuracy and productivity (ranked highest factor)
- Skill level with programming tools and languages
- Communication overhead (Brooks's Law effect)
- Coordination challenges in larger teams
- Team cohesion impacts COCOMO scale factors (TEAM)



Time & Scheduling Factors

How time is allocated affects costs—both in agile and traditional environments.

- Shorter, iterative cycles reduce large uncertainty
- Delays in one stage influence downstream tasks
- Verification & validation effort distribution differs (Agile vs Traditional)
- Changing deadlines add cost pressures



Organizational Maturity



Organizations with well-established processes experience lower cost variation.

- Use of standards (CMMI, structured processes)
- Experience handling Agile in acquisition environments
- Better risk identification and management
- Clear communication between teams and stakeholders

Relevant insight from Agile acquisition study: Organizations lacking maturity struggle with estimation consistency.

Summary: Combined Cost Influencing Factors

Software costs are shaped by many dimensions that interact across the product, team, and organization.

- Larger and more complex products cost more
- Experienced teams estimate and deliver more accurately
- Organizational maturity enhances predictability
- Technical complexity drives testing and integration cost
- Agile processes reduce uncertainty but require team alignment
- Research shows experience, complexity, time, and size as core drivers

All factors must be considered for realistic software cost estimation.

Proven Techniques for Estimating Project Budgets



What Are Budget Estimation Techniques?

Estimation techniques help teams predict cost and effort more accurately before development begins. They provide structure so teams don't rely on guesswork. Different techniques suit different levels of detail and project types.

- Improve prediction of effort and cost
- Reduce uncertainty during planning
- Support informed decision-making



Why These Techniques Matter



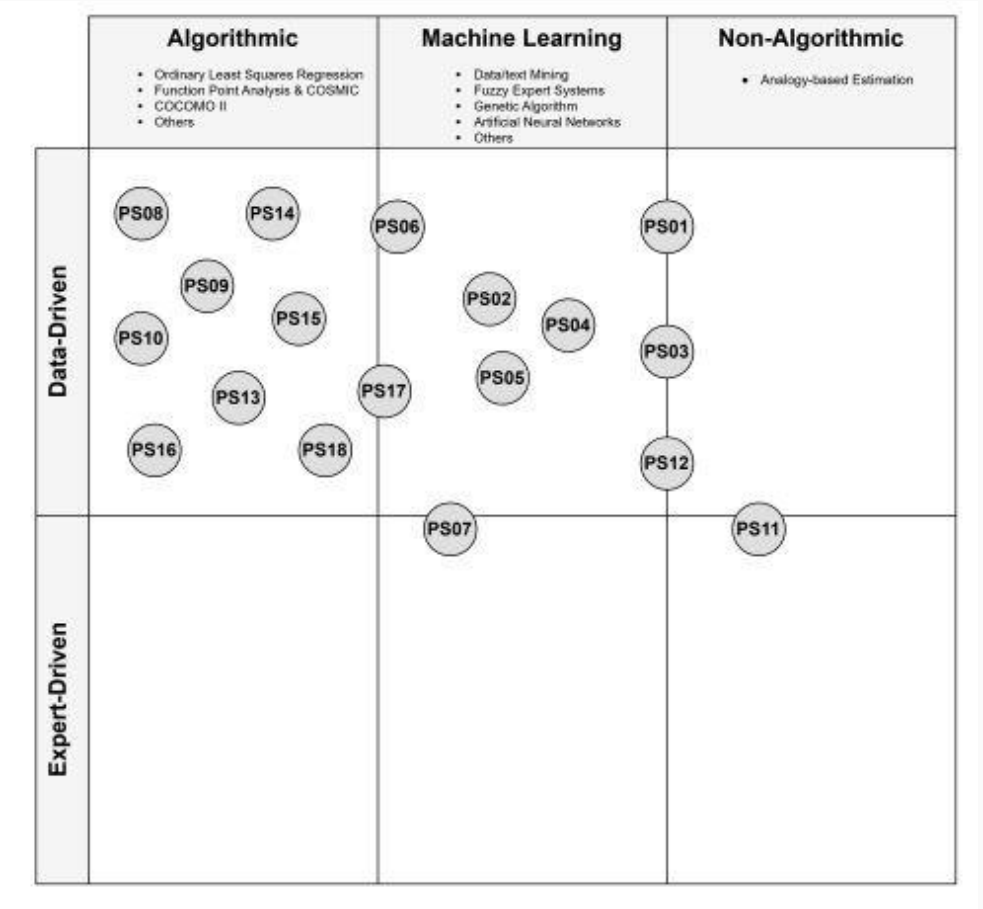
Using proven estimation methods helps teams avoid underestimation and project overruns. Teams gain clearer understanding of workload and risks. These techniques also improve planning confidence for stakeholders.

- Avoid unrealistic commitments
- Identify risk early
- Improve communication with stakeholders

Categories of Estimation Techniques

Agile estimation methods fall into several major categories. Each category offers a different way to measure effort or size. Understanding them helps teams choose the right method for budgeting and planning.

- Story-based / relative estimation
- Time-based estimation
- Probabilistic / flow forecasting
- Expert-judgment methods
- Hybrid / model-assisted approaches



Top-Down Estimation



What Is Top-Down Estimation?

Top-down estimation starts with a big-picture view of the project and gives a quick overall effort estimate. It is used early when details are not fully known.

- Estimate from overall system
- Fast and simple
- Used early in planning
- Uses T-shirt sizing & high-level Story Points



Types of Top-Down Estimation



Top-down estimation uses visible features of the product to estimate effort based on size groups and similarity.

- T-Shirt Estimating (group items into S, M, L, XL)
- Function Point Estimating
- Velocity-Based Estimation

1. T-Shirt Estimation

What Is T-Shirt Sizing?

T-shirt sizing assigns categories like S, M, L, and XL to represent effort levels. It is a coarse estimation method used when details are not clear. Teams often use it for epics or early-stage planning.

- Uses S–XL categories
- Very simple and fast
- Ideal for high-level estimation



1. T-Shirt Estimation



Why Teams Use T-Shirt Sizing

Teams use T-shirt sizing because it avoids over-analysis and quickly shows which items are large or small. It helps stakeholders understand scope without technical detail. Later, teams convert sizes into Story Points.

- Quick scope comparison
- Good for discussions with stakeholders
- Helps identify large epics early

1. T-Shirt Estimation - Example

Concept

- Requirements/stories are grouped into size categories (S, M, L, XL).
- Each size is assigned an estimated effort (e.g., in 6-hour days).
- Total effort = Size Effort × Quantity of Stories.

AMS-FR Case Study: Sample T-Shirt Estimation

User Stories

1. **FR1**: Capture student facial image during check-in → *Small*
2. **FR2**: Validate student identity using facial recognition model → *Medium*
3. **FR3**: Auto-generate attendance status and sync to dashboard → *Medium*

Assigned Effort

- Small = **1 day**
- Medium = **3 days**

Effort Calculation

Story	Size	Effort
FR1	S	1 × 1 = **1 day**
FR2	M	1 × 3 = **3 days**
FR3	M	1 × 3 = **3 days**

Total Estimated Effort = 1 + 3 + 3 = 7 days

2. Velocity-Based Estimation

What Is Velocity-Based Release Planning?

Velocity measures how many Story Points a team completes each sprint. Release planning uses this velocity to forecast how long it will take to finish the product backlog. It becomes more accurate after several sprints.

- Based on completed Story Points
- Used to predict release timelines
- Improves as team consistency grows



2. Velocity-Based Estimation



Why Teams Use Velocity

Teams use velocity because it provides data-driven forecasting instead of guesswork. It allows predictable budgeting and helps identify when a scope change affects delivery. Velocity also supports continuous improvement.

- Helps forecast future work
- Tracks changes in team performance
- Supports realistic planning conversations

2. Velocity-Based Estimation - Example

Example (Given in Text)

If a team delivers **20 story points per Timebox**, and the next Increment has **5 Timeboxes**, expected output:
20 × 5 = 100 story points

AMS-FR Case Study: Sample Velocity Calculation

User Stories

Story	Description	Story Points
FR1	Capture facial image	3
FR2	Facial recognition matching	8
FR3	Auto-generate attendance status	5
FR4	Sync to dashboard + reporting	5

Team's Past Velocity (Observed Over 3 Timeboxes)

- TB1: 18 pts
- TB2: 22 pts
- TB3: 20 pts

Average Velocity = (18 + 22 + 20) ÷ 3 = 20 story points per Timebox

---

Concept

- ❖ Velocity = Average number of story points a team delivers per Timebox.
- ❖ Used to forecast how much work can be completed in upcoming Timeboxes

- ❑ Velocity is unique to each team and cannot be compared across teams.
- ❑ Changes in team composition require recalculating velocity.
- ❑ For new teams, velocity is estimated during Timebox Planning by sizing and filling the Timebox until capacity is reached.

Team's Past Velocity (Observed Over 3 Timeboxes)

- TB1: 18 pts
- TB2: 22 pts
- TB3: 20 pts

Average Velocity = (18 + 22 + 20) ÷ 3 = 20 story points per Timebox

---

Forecast for Next Timebox

Total AMS-FR story points = **3 + 8 + 5 + 5 = 21 points**

With a velocity of **20 points**, the forecast is:

- ✔ **The team can deliver ~20 points in the next Timebox**
- ✔ Remaining **1 point** moves to the next Timebox or is absorbed if capacity allows

---

3. Function Point Estimation

What Is a Function Point?

A Function Point measures the amount of functionality a system provides to the user. It counts things like inputs, outputs, and data files to show how big the system is. It does not depend on programming language.

- Measures system size
- Based on user-visible functions
- Independent of technology



3. Function Point Estimation



Why Teams Use Function Points

Teams use Function Points to estimate work early before coding begins. FP helps them understand system size and plan cost or effort. It gives a clear and consistent way to compare projects.

- Measures system size
- Based on user-visible functions
- Independent of technology

3. Function Point Estimation



Components of Function Points

FP is calculated by identifying different types of system functions. Each function type contributes a certain number of points depending on complexity. These components reflect how data and processes flow in the system.

- External Inputs (EI)
- External Outputs (EO)
- External Inquiries (EQ)
- Internal Logical Files (ILF)
- External Interface Files (EIF)

3. Function Point Estimation - Example

Case Study: AMS-FR (Attendance Management System – Facial Recognition)

Step 1: Identify System Boundary

The AMS-FR system includes:

- Student facial check-in
- Facial recognition validation
- Attendance recording
- Analytics dashboard & reports
- Integration with student database

Step 2: Identify Function Types

Using standard IFPUG components:

Function Type	Description in AMS-FR
EI (External Input)	Data entering the system
EO (External Output)	Data leaving the system with processing
EQ (External Inquiry)	Data retrieval with minimal processing
ILF (Internal Logical File)	Internal data maintained by system
EIF (External Interface File)	External data used but not maintained

Step 3: Map AMS-FR Functions to FP Components

External Inputs (EI)

No	Function	Complexity
EI1	Capture student facial image during check-in	Medium
EI2	Lecturer manually update attendance (override)	Low

Total EI = 2

External Outputs (EO)

No	Function	Complexity
EO1	Auto-generate attendance status	Medium
EO2	Generate attendance analytics report	High

Total EO = 2

3. Function Point Estimation - Example

External Inquiries (EQ)

No	Function	Complexity
EQ1	Lecturer view student attendance record	Low
EQ2	Search attendance by date/class	Low

Total EQ = 2

Internal Logical Files (ILF)

No	Data File	Complexity
ILF1	Student Profile Database	Medium
ILF2	Attendance Records	Medium
ILF3	Facial Feature Templates	High

Total ILF = 3

External Interface Files (EIF)

No	External File	Complexity
EIF1	University Student Information System	Medium

Total EIF = 1

Step 4: Apply Standard FP Weights

Component	Low	Medium	High
EI	3	4	6
EO	4	5	7
EQ	3	4	6
ILF	7	10	15
EIF	5	7	10

3. Function Point Estimation - Example

Step 5: Calculate Unadjusted Function Points (UFP)

EI Calculation

- EI1 (Medium): 4
- EI2 (Low): 3

  EI Total = 7

EO Calculation

- EO1 (Medium): 5
- EO2 (High): 7

  EO Total = 12

EQ Calculation

- EQ1 (Low): 3
- EQ2 (Low): 3

  EQ Total = 6

ILF Calculation

- ILF1 (Medium): 10
- ILF2 (Medium): 10
- ILF3 (High): 15

  ILF Total = 35

EIF Calculation

- EIF1 (Medium): 7

  EIF Total = 7

Step 6: Total Unadjusted Function Points

$$\mathbf{UFP} = 7 + 12 + 6 + 35 + 7 = \boxed{67 \text{ Function Points}}$$

(Optional – for Advanced Students)

You may later introduce:

- **Value Adjustment Factor (VAF)**
- **Adjusted FP = UFP × VAF**
- Conversion to effort:

  `Effort (days) = FP × productivity rate (e.g. 0.5 day/FP)`

Example:

$$67 \times 0.5 = 33.5 \text{ person-days}$$

Bottom-Up Estimation



What Is Bottom-Up Estimation?

Bottom-up estimation starts by breaking work into small tasks and estimating each one. It is used when the team has clear details of the work.

- Estimate from small tasks
- More accurate
- Used later when details are known
- Uses task breakdown & detailed sizing



Types of Bottom-Up Estimation



Bottom-up estimation breaks work into smaller pieces and estimates each one separately.

- Work Breakdown Estimating (estimate each task)
- Product Breakdown Estimating

1. Product Breakdown Estimating

What is Product Breakdown Estimating

Product Breakdown Estimating divides the system into smaller components or sub-products. Each component is then estimated separately. This approach is helpful when dealing with large or complex systems.



- Breaks work into smaller tasks
- Estimates each task individually
- Adds tasks to get total effort

1. Product Breakdown Estimating



Why Teams Use Product Breakdown Estimating

Teams use Product Breakdown Estimating to simplify large features into manageable pieces. It helps ensure no major part of the system is forgotten. This method improves clarity and planning accuracy for big projects.

- Simplifies complex systems
- Ensures complete coverage of components
- Improves planning for large features

2. Work Breakdown Estimating

What Is Work Breakdown Estimating?

Work Breakdown Estimating breaks a feature into small, clear tasks and estimates each task separately. It helps teams understand the detailed work involved before development begins. The total effort is calculated by adding all task estimates together.



- Breaks work into smaller tasks
- Estimates each task individually
- Adds tasks to get total effort

2. Work Breakdown Estimating



Why Teams Use Work Breakdown Estimating

Teams use Work Breakdown Estimating because it gives more accurate estimates and reduces missed work. It also helps teams see what needs to be done in detail. This method is especially useful during sprint planning.

- Produces more accurate estimation
- Helps identify all required tasks
- Useful for detailed sprint planning

2. Work Breakdown Estimating - Example

Case Study: AMS-FR (Attendance Management System with Facial Recognition)

Step 1: Identify Major Product Components

The AMS-FR system is decomposed into major sub-products:

1. Facial Recognition Module
2. Attendance Management Module
3. Analytics & Reporting Module
4. User Interface Module
5. System Integration Module

Step 2: Break Each Component into Sub-Tasks

1. Facial Recognition Module

Sub-Component	Estimated Effort (Days)
Capture facial image	2
Facial feature extraction	3
Face matching & validation	4
Error handling & accuracy tuning	2
Subtotal	**11 days**

2. Attendance Management Module

Sub-Component	Estimated Effort (Days)
Auto-mark attendance	2
Manual attendance override	1
Attendance status rules	2
Attendance storage logic	2
Subtotal	**7 days**

3. Analytics & Reporting Module

Sub-Component	Estimated Effort (Days)
Attendance analytics computation	3
Dashboard visualisation	3
Report export (PDF/Excel)	2
Subtotal	**8 days**

2. Work Breakdown Estimating - Example

4. User Interface Module

Sub-Component	Estimated Effort (Days)
Student check-in interface	2
Lecturer dashboard UI	3
Admin configuration screens	2
Subtotal	**7 days**

5. System Integration Module

Sub-Component	Estimated Effort (Days)
Integration with student database	2
Authentication & access control	2
System testing & integration validation	2
Subtotal	**6 days**

Step 3: Calculate Total Estimated Effort

Module	Effort (Days)
Facial Recognition	11
Attendance Management	7
Analytics & Reporting	8
User Interface	7
System Integration	6
Total Estimated Effort	**39 days**

Team Size	Duration (Approx.)
1 person	39 days
2 people	19–20 days
3 people	13 days
4 people	~10 days

2. Work Breakdown Estimating - Example

Agile / Timebox Context (Recommended Explanation)

If:

- 1 sprint = 2 weeks = **10 working days**
- Team size = **3 members**

Then:

- Team capacity per sprint = 3 × 10 = **30 person-days**
- Total effort = **39 person-days**

➡️ **System can be completed in about 1.5 sprints**



THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

www.utm.my